



BITS Pilani
Pilani Campus

Deploying Microservices

Prof. Akanksha Bharadwaj
Asst. Professor, CSIS Department



BITS Pilani
Pilani Campus



SE ZG583, Scalable Services

Lecture No. 12

Agenda



Deploying Microservices

Service startup

Running multiple instances

Adding load balancer

Service to host models

- Single Service Instance to Host
- Multiple static Service Per Host
- Multiple scheduled services per host

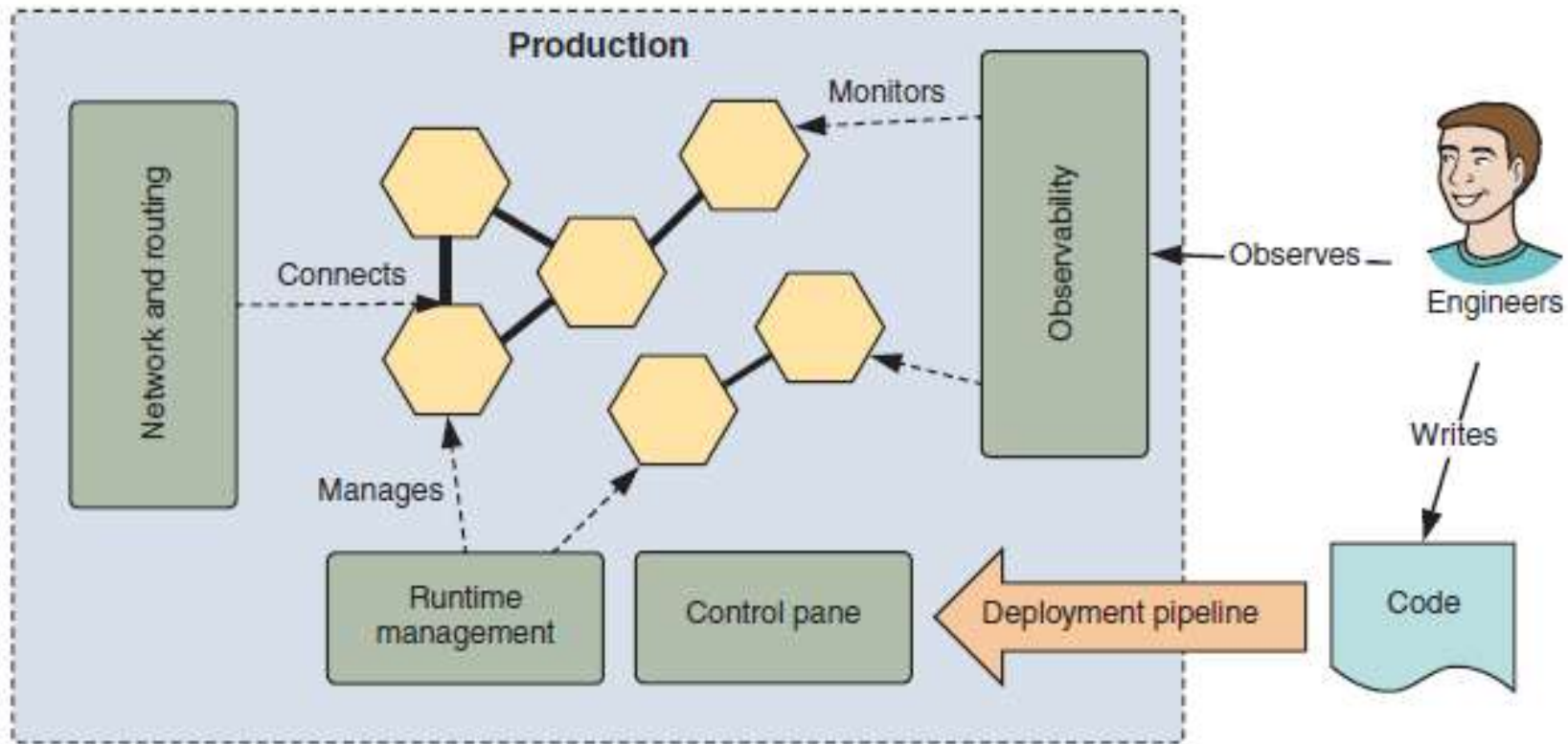
Challenges in deploying Microservices



Installing microservices can be complex due to their distributed nature. Here are the key challenges:

- **Dependency Management:** Microservices often rely on multiple libraries, frameworks, and external services.
- **Configuration Complexity:** Each microservice may require its own configuration for databases, APIs, or environments (dev, staging, production).
- **Networking and Communication:** Microservices communicate over networks, introducing issues like latency, timeouts, and service discovery.
- **Containerization and Orchestration:** Most microservices use containers (e.g., Docker) and orchestration tools (e.g., Kubernetes).
- **Monitoring and Logging:** With many services running independently, tracking performance, errors, and logs becomes challenging.

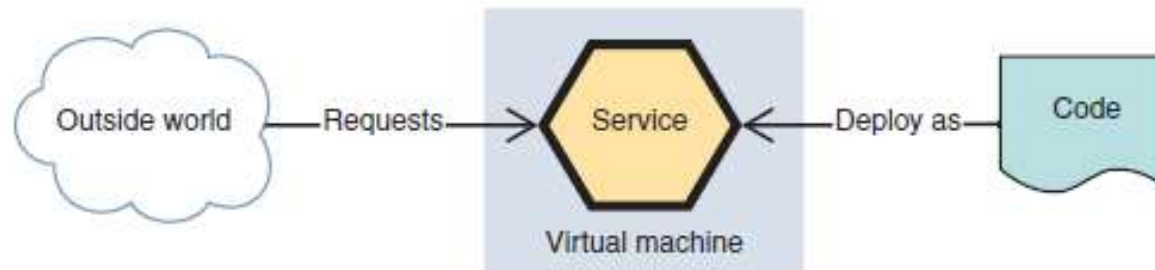
Features of a microservice production environment



Service Startup



- You need to take your code, get it running on a virtual machine, and make it accessible from the outside world



Service Startup

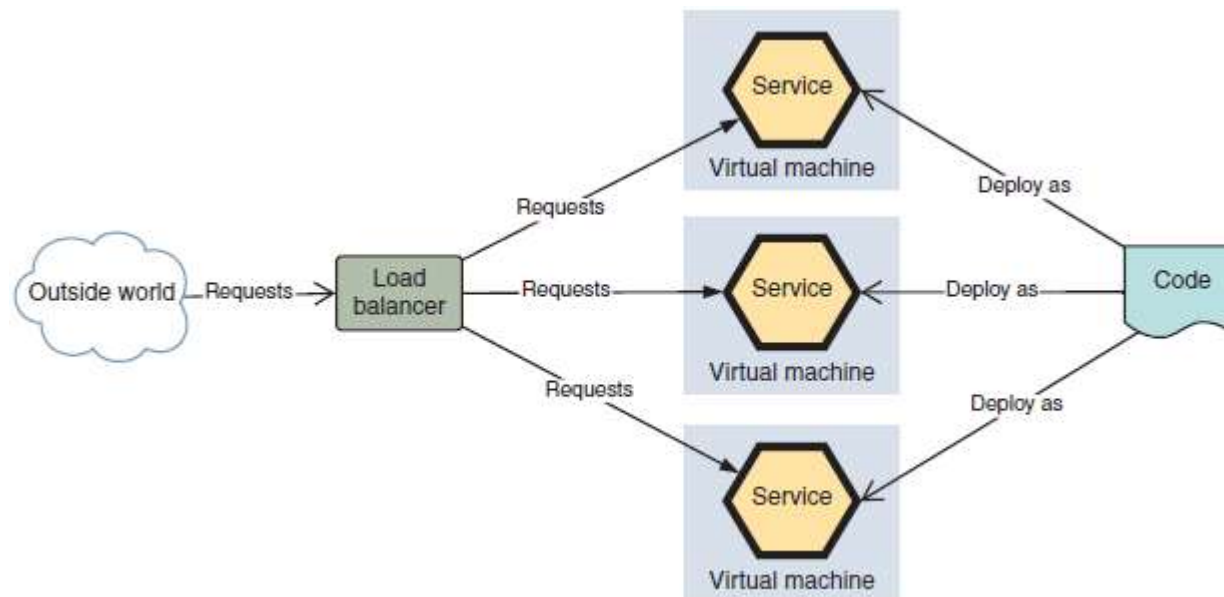


- Installs binary dependencies required to run the application
- Downloads your service code from Github or any other repository
- Installs that code's dependencies
- Configure a supervisor if required

Run multiple instances of a service

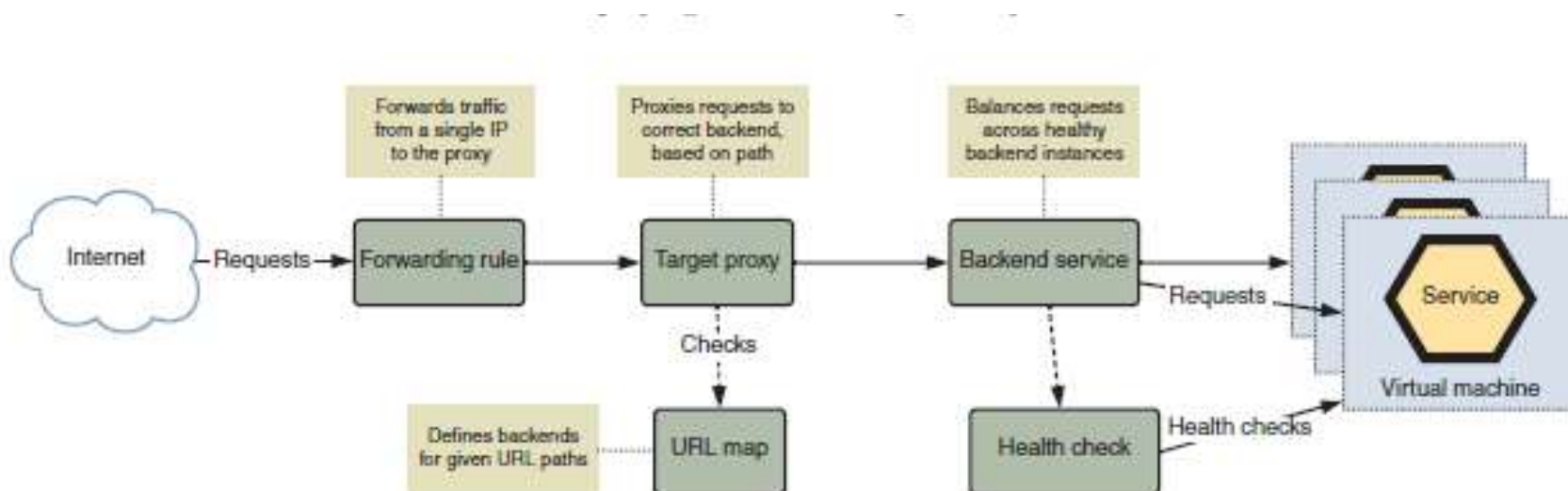


- Think about Scale horizontally
- Deploy the redundancy



Adding a load balancer

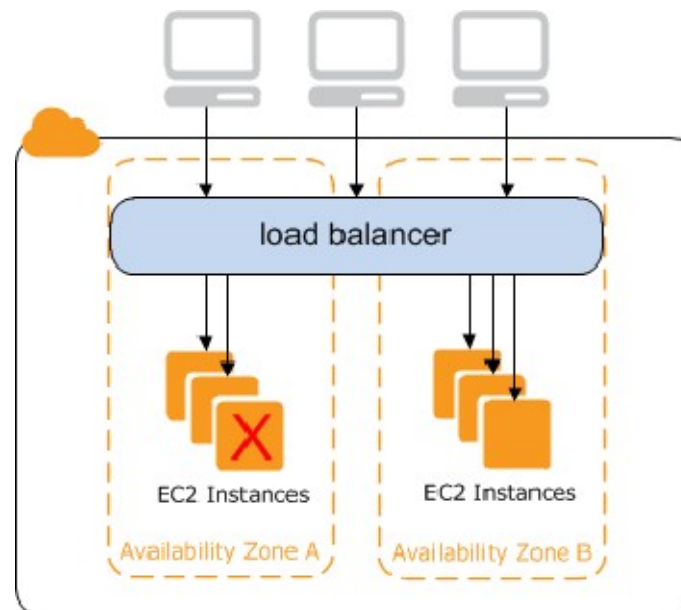
- The load balancer uses routing rules, proxies, and maps to forward requests from the outside world to a set of healthy service instances.



AWS: What is a Classic Load Balancer?



- Elastic Load Balancing automatically distributes your incoming traffic across multiple targets, such as EC2 instances, containers, and IP addresses, in one or more Availability Zones.





BITS Pilani
Pilani Campus



Models for Deploying Services

Multiple Service Instances per Host Pattern



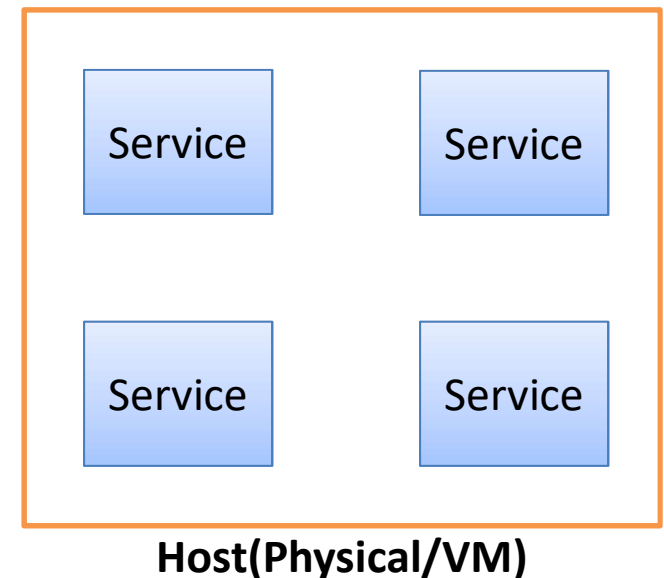
Deploying multiple services to a single host in production is synonymous with deploying multiple services to a local dev workstation or laptop.

Benefits:

- Attractive from a host management point of view
- Cost

Challenges:

- Monitoring
- Isolation of instances
- Resource consumption



Single Service Instance per Host Pattern



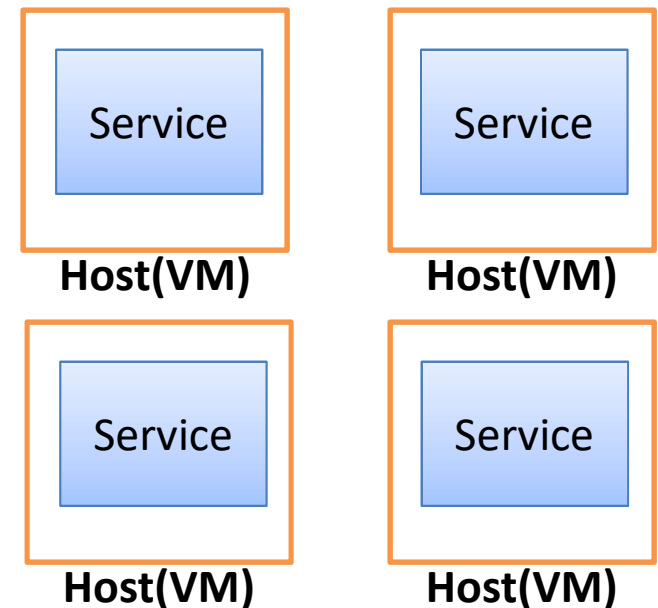
We can avoid the side effects of multiple services living on a single host, by making monitoring and remediation much simpler

Benefits

- Service instance Isolation
- Easy to manage, monitor and deploy
- No conflicting resource requirements

Drawbacks

- Less efficient resource utilization
- cost



Service Instance per Container Pattern



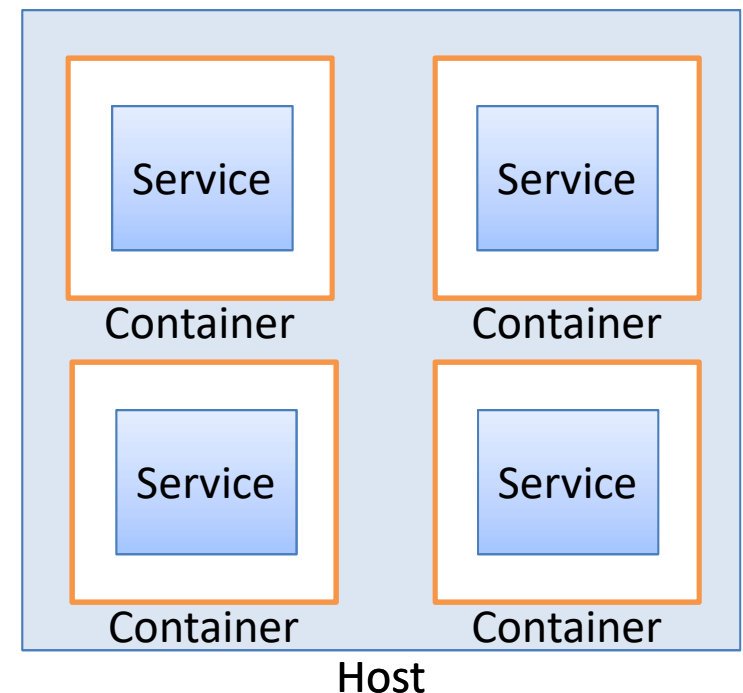
Package the service as a container image and deploy each service instance as a container

Benefits

- Easy to scale up and down a service.
- Isolation
- Limit the resource utilization by a service instance
- Faster deployment

Drawbacks

- Infrastructure for deploying containers is not rich



Best Deployment Strategy



3 main practical and popular approaches to deploy Microservices:

- Deploy each Microservice instance on a separate Virtual Machine
- Deploy each Microservice instance as a (Docker) container on Kubernetes
- Deploy each Microservice as a Serverless Function

Running a single service on local machine



- Open the service that you have created
- Use an appropriate command in the command prompt to run the service
- Use local browser/postman application to check the functioning

Dockerfile



- Docker can build images automatically by reading the instructions from a Dockerfile

Basic Docker commands:

- FROM
- RUN
- ENV
- COPY
- EXPOSE
- ENTRYPOINT
- CMD
- VOLUME
- WORKDIR
- LABEL
- ADD
- ARG

Dockerfile best practices



- Pick the right base image
- Keep the number of steps in the Dockerfile to minimum
- Control the structure of your dockerfile

Docker Compose



- Compose is a tool for defining and running multi-container Docker applications.

Basic Structure of Docker compose file

- version
- services
- build
- command
- ports
- volumes
- links
- image
- environment
- restart
- depends_on

Run multiple services on a container



If you need to run more than one service within a container, you can accomplish this in a few different ways.

- Use wrapper script
- Use a process manager like supervisord

Run a single service using Docker desktop



- Create a dockerfile
- Build a Docker image
- Run in Docker
- Check your running application
- Observe the running container
- Stop the container

Self Study



-
- <https://docs.aws.amazon.com/elasticloadbalancing/latest/classic/elb-getting-started.html>
 - <https://docs.docker.com/>

References



- Chapter8, Microservices in Action
- <https://docs.aws.amazon.com/elasticloadbalancing>